LOW-LEVEL DESIGN (LLD)

RAG Customer Support Assistant

1. MODULE DESIGN

1.1 Document Loader Module

File: ingestion/document_loader.py

```
Class: DocumentLoader
Method: load_pdf(file_path)
 ├─ Input: PDF file path (string)
 ├─ Output: List of documents (one per page)
 └─ Logic:
   1. Open PDF using PyMuPDF
   2. Loop through each page
   3. Extract text
   4. Create document object with metadata
   5. Return list of documents
```

1.2 Chunker Module

File: ingestion/chunker.py

```
Class: DocumentChunker
Settings:
- Chunk size: 512 tokens
- Overlap: 50 tokens
Method: chunk_documents(documents)
 ├─ Input: List of documents
 ├─ Output: List of chunks
 └─ Logic:
   1. For each document:
      - Split text into 512-token pieces
```

2. DATA STRUCTURES

2.1 Core Objects

```
# Document
{
    doc_id: "shopease_p1",
    source_file: "shopease.pdf",
    page_number: 1,
    text: "extracted text...",
    metadata: {...}
}

# Chunk
{
    chunk_id: "shopease_p1_chunk_0",
    text: "512 token text...",
    metadata: {"source": "shopease.pdf", "page": 1},
    token_count: 487
}

# Retrieved Chunk
{
    text: "chunk content...",
    source: "shopease.pdf",
    page: 2,
    similarity_score: 0.882,
    rank: 1
}

# Graph State (flows between nodes)
```

```
{
    user_query: "How do I track order?",
    intent: "KNOWLEDGE",
    retrieved_chunks: [...],
    retrieval_confidence: 0.88,
    llm_answer: "To track your order...",
    final_answer: "...",
    sources: ["shopease.pdf (Page 2)"],
    escalation_triggered: False,
    ticket_id: None
}

# Ticket
{
    ticket_id: "TKT-20250115-7A2B",
    status: "OPEN",
    priority: "HIGH",
    user_query: "...",
    escalation_reason: "...",
    created_at: "2025-01-15 14:30:22",
    human_response: None
}
```

3. LANGGRAPH WORKFLOW

3.1 Node Functions

```
NODE 1: input_node
├─ Validate query
├─ Initialize session
└─ Set next_action = "classify"
```

NODE 2: classify_node

├─ Check if greeting → intent = "GREETING"

├─ Check if escalation request → intent = "ESCALATE"

├─ Check if complex → intent = "COMPLEX"

└─ Default → intent = "KNOWLEDGE"

NODE 3: direct_response_node

└─ Return greeting message

NODE 4: retrieve_node

├─ Search ChromaDB

├─ Get top-4 chunks

├─ If score < 0.35 → next_action = "hitl"

└─ If score >= 0.35 → next_action = "generate"

NODE 5: generate_node

├─ Build prompt with context

├─ Call LLM API

└─ Get answer

NODE 6: confidence_node

├─ Check for uncertainty phrases

├─ If uncertain → next_action = "hitl"

└─ If confident → next_action = "output"

NODE 7: hitl_node

├─ Create ticket

├─ Get human response

└─ Set final_answer = human response

```
NODE 8: output_node
├─ Extract sources
├─ Update conversation history
└─ Return final answer
```

4. ROUTING LOGIC

4.1 Intent Classification

```
IF "hello" OR "hi" in query AND words <= 5:
    intent = "GREETING"

ELSE IF "speak to human" OR "real agent" in query:
    intent = "ESCALATE"

ELSE IF question_marks > 2:
    intent = "COMPLEX"

ELSE:
    intent = "KNOWLEDGE"
```

4.2 Confidence Routing

```
After retrieval:
IF top_score < 0.35:
    → Route to HITL

After LLM generation:
IF "I don't know" in answer OR words < 8:
    → Route to HITL
ELSE:
    → Route to OUTPUT
```

5. HITL WORKFLOW

```
User asks question
  ↓
System can't answer confidently
  ↓
Create Ticket
  ├─ ID: TKT-20250115-7A2B
  ├─ Priority: Calculate based on reason
  └─ Save to tickets.json
  ↓
Display to Agent (CLI)
  ├─ Show: Query, Reason, Context
  └─ Prompt for response
  ↓
Agent Types Response
  ↓
Update Ticket
  ├─ Status: RESOLVED
  └─ Save response
  ↓
Return to User
```

6. ERROR HANDLING

ERROR: PDF not found

ACTION: Skip file, log warning

ERROR: No chunks retrieved

ACTION: Escalate to HITL

ERROR: LLM API fails

ACTION: Return default message, escalate to HITL

ERROR: ChromaDB connection fails

ACTION: Critical error, stop system

ERROR: Empty query

ACTION: Prompt user again

7. ALGORITHM PSEUDOCODE

7.1 Main Query Processing

```
FUNCTION process_query(user_query):

    // Step 1: Initialize
    state = create_initial_state(user_query)

    // Step 2: Classify intent
    state = classify_intent(state)

    // Step 3: Route based on intent
    IF state.intent == "GREETING":
        state.final_answer = "Hello! How can I help?"
        RETURN state

    IF state.intent == "ESCALATE":
        state = escalate_to_human(state)
        RETURN state

    // Step 4: Retrieve from knowledge base
    state = retrieve_chunks(state)
```

```
    IF state.retrieval_confidence < 0.35:
        state = escalate_to_human(state)
        RETURN state

    // Step 5: Generate answer
    state = generate_llm_answer(state)

    // Step 6: Check confidence
    IF state.llm_answer contains "I don't know":
        state = escalate_to_human(state)
        RETURN state

    // Step 7: Format output
    state.final_answer = state.llm_answer
    state = add_sources(state)

    RETURN state
```

7.2 Retrieval Algorithm

```
FUNCTION retrieve_chunks(query):

    // Convert query to vector
    query_vector = embedding_model.encode(query)

    // Search database
    results = chromadb.query(
        query_vector,
        top_k = 4
    )
```

```
    // Calculate scores
    chunks = []
    FOR each result in results:
        score = 1 - (result.distance / 2)
        chunk = {
            text: result.document,
            source: result.metadata.source,
            page: result.metadata.page,
            score: score
        }
        chunks.append(chunk)

    // Check confidence
    top_score = chunks[0].score
    is_confident = (top_score >= 0.35)

    RETURN chunks, top_score, is_confident
```

8. PERFORMANCE SPECS

Target Metrics:

```
├─ Response time: < 4 seconds (95th percentile)
├─ Retrieval time: < 100ms
├─ LLM generation: < 1 second
├─ Accuracy: > 90%
└─ HITL rate: 15-20%
```

Optimization:

```
├─ Batch embedding: 32 chunks at once
├─ Cache frequent queries
└─ Use HNSW indexing in ChromaDB
```

9. FILE STRUCTURE

```
rag_support_assistant/
│
├── config.py              # All settings
├── main.py                # Entry point
│
├── ingestion/
│   ├── document_loader.py     # Load PDFs
│   ├── chunker.py             # Split text
│   └── embedder.py            # Create vectors
│
├── storage/
│   └── vector_store.py        # ChromaDB operations
│
├── retrieval/
│   └── retriever.py           # Search chunks
│
├── llm/
│   └── llm_client.py          # Call Groq API
│
├── workflow/
│   ├── state.py               # State definition
│   ├── nodes.py               # Node functions
│   ├── graph.py               # Build workflow
│   └── router.py              # Routing logic
│
├── hitl/
│   └── hitl_manager.py        # Human escalation
│
└── data/
```

```
├── documents/          # PDFs go here
└── chroma_db/          # Vector database
```

CONCLUSION

This LLD provides clear implementation details for:

- ✅ All 7 modules with inputs/outputs
- ✅ Complete workflow with 8 nodes
- ✅ Routing logic and decision points
- ✅ HITL escalation process
- ✅ Error handling strategy
- ✅ Key algorithms in pseudocode